Informe de proyecto 1 Arquitectura de Software

Tomas Silva
Juan Pabon Vargas
Alexander Aponte



Pontificia Universidad
JAVERIANA
Colombia

Andres Armando Sanchez Martin

Pontificia Universidad Javeriana

20 de Octubre 2026

Índice

Marco conceptual....2
Introducción....2
Conceptos arquitectónicos....2
Herramientas y tecnologías....3
Modelo de datos....3
Relaciones....4
Diseño....5
Procedimiento....10
1. Preparación del entorno....10
2. Diseño de la base de datos....12
3. Creación del proyecto MVC y arquitectura en capas....14
4. Capa de modelo: Database First con Scaffold-DbContext....15
5. Configuración: conexión, inyección de dependencias y Swagger....19
6. Capa de acceso a datos: patrón DAO con repositorios....21
7. ViewModels: no exponer las entidades....24
8. Controladores MVC y API REST....27
9. Vistas Razor....30
10. Integridad y manejo de errores....32
11. Trabajo en equipo con Git....34
12. Pruebas, despliegue y resumen de decisiones....35
Conclusiones y lecciones....39
Referencias....39

Marco conceptual

Introducción

Este laboratorio implementa un monolito en .NET que gestiona el CRUD de un modelo de datos de personas. Combina el patrón arquitectónico MVC con el patrón de acceso a datos DAO (en la práctica, repositorios), sobre una base de datos relacional en SQL Server y con una API REST documentada con Swagger.

Conceptos arquitectónicos

Arquitectura monolítica. Es un estilo en el que toda la aplicación (vistas, lógica de negocio, acceso a datos y API) se construye, empaqueta y despliega como una sola unidad. Es sencilla de desarrollar, probar y desplegar, lo que la hace adecuada para un laboratorio, aunque escala y evoluciona con más dificultad que una arquitectura distribuida.

Patrón MVC (Modelo-Vista-Controlador). Separa la aplicación en tres responsabilidades:

- **Modelo:** representa los datos y las reglas del dominio (las entidades).
- **Vista:** es la interfaz que ve el usuario (vistas Razor).
- **Controlador:** recibe las peticiones, coordina al modelo y decide qué vista o respuesta devolver.

Patrón DAO (Data Access Object). Encapsula el acceso a la base de datos en una capa independiente, de modo que los controladores no conocen los detalles de persistencia. En este laboratorio se materializa con interfaces y repositorios que exponen las operaciones CRUD. Esto reduce el acoplamiento y facilita cambiar la fuente de datos o hacer pruebas.

CRUD. Son las cuatro operaciones básicas sobre los datos: *Create, Read, Update, Delete*. Se exponen como endpoints REST y como vistas.

Herramientas y tecnologías

De todas las herramientas, hasta ahora solo se ha usado SQL Server; el resto se utilizará en las etapas siguientes.

Microsoft SQL Server 2025 Express (instalado). Es el sistema gestor de bases de datos relacionales de Microsoft. En él se creará la base de datos **`persona_db`**, que almacenará las tablas del modelo.

SQL Server Management Studio (SSMS). Es la herramienta gráfica para crear la base de datos, ejecutar los scripts DDL (estructura) y DML (datos) y administrar las tablas.

.NET 10 y ASP.NET Core. NET es la plataforma de desarrollo de Microsoft, multiplataforma y de código abierto. ASP.NET Core es su framework para aplicaciones web y APIs, e incluye soporte nativo para MVC, inyección de dependencias y configuración mediante archivos como **`appsettings.json`**.

Entity Framework Core (EF Core). Es un ORM (mapeador objeto-relacional) que permite trabajar con la base de datos mediante clases C# en lugar de SQL directo. Se usará el enfoque *Database First*: como la base de datos ya existe, el comando **`Scaffold-DbContext`** generará las clases entidad y el **`DbContext`.** Los paquetes previstos son **`Microsoft.EntityFrameworkCore`**, **`Microsoft.EntityFrameworkCore.SqlServer`** y **`Microsoft.EntityFrameworkCore.Tools`.**

API REST. Es un estilo arquitectónico para servicios web basado en HTTP. Los recursos se identifican con URLs y se manipulan con los verbos estándar: **`GET`** (consultar), **`POST`** (crear), **`PUT`** (actualizar) y **`DELETE`** (eliminar). Los datos suelen intercambiarse en formato JSON.

Swagger / OpenAPI 3. Es una especificación para describir APIs REST. Swagger UI genera una página interactiva donde se documentan y prueban los endpoints sin herramientas externas.

Visual Studio Community 2026. Es el entorno de desarrollo integrado (IDE) con el que se creará el proyecto, se gestionarán los paquetes NuGet, se explorará la base de datos y se depurará. Requiere las cargas de trabajo de desarrollo ASP.NET y web y de almacenamiento y procesamiento de datos.

Git y GitHub. Git es el sistema de control de versiones y GitHub el servicio remoto donde se alojará el repositorio **`personapi-dotnet`**. Permite el trabajo colaborativo del equipo y marcar la versión entregable mediante un *tag*.

Modelo de datos

El laboratorio trabaja con un modelo relacional de cuatro entidades, que es la base para generar las entidades con Entity Framework y los CRUD de cada tabla.

Relaciones

Entidad	Propósito	Columnas
persona	Datos básicos de cada individuo	cc (PK), nombre, apellido, genero (M/F), edad
profesion	Catálogo de profesiones	id (PK), nom, des
estudios	Tabla intermedia que registra qué persona cursó qué profesión	id_prof, cc_per, fecha, univer
telefono	Números de contacto	num (PK), oper (operador), duenio (referencia a persona)

- **persona – telefono (1:N):** una persona puede tener varios teléfonos y cada teléfono pertenece a una sola persona.
- **persona – profesion (N:M):** una persona puede tener varias profesiones y una profesión puede estar asociada a varias personas. Se resuelve mediante la tabla de estudios**.**

Diseño

El sistema está basado en una arquitectura por capas usando el patrón MVC siguiendo el siguiente contexto

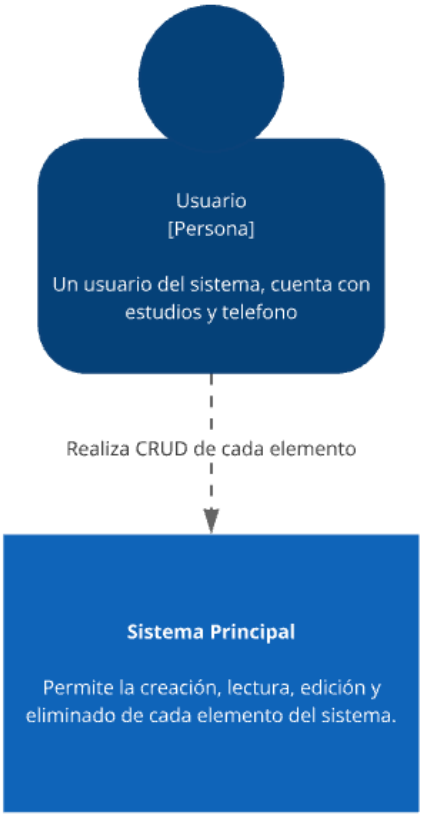


Diagrama de contexto para sistema principal
Ultima modificación: 2026-10-21

Diagrama 1, Diagrama C4 de contexto del sistema

Se cuenta con un único usuario el cual se conecta y realiza operaciones CRUD para cada una de las entidades en el sistema.

El sistema como general consiste en dos partes, el sistema general, que contiene controladores, vistas, modelos entre otros, y la base de datos MS SQL, como se muestra a continuación:

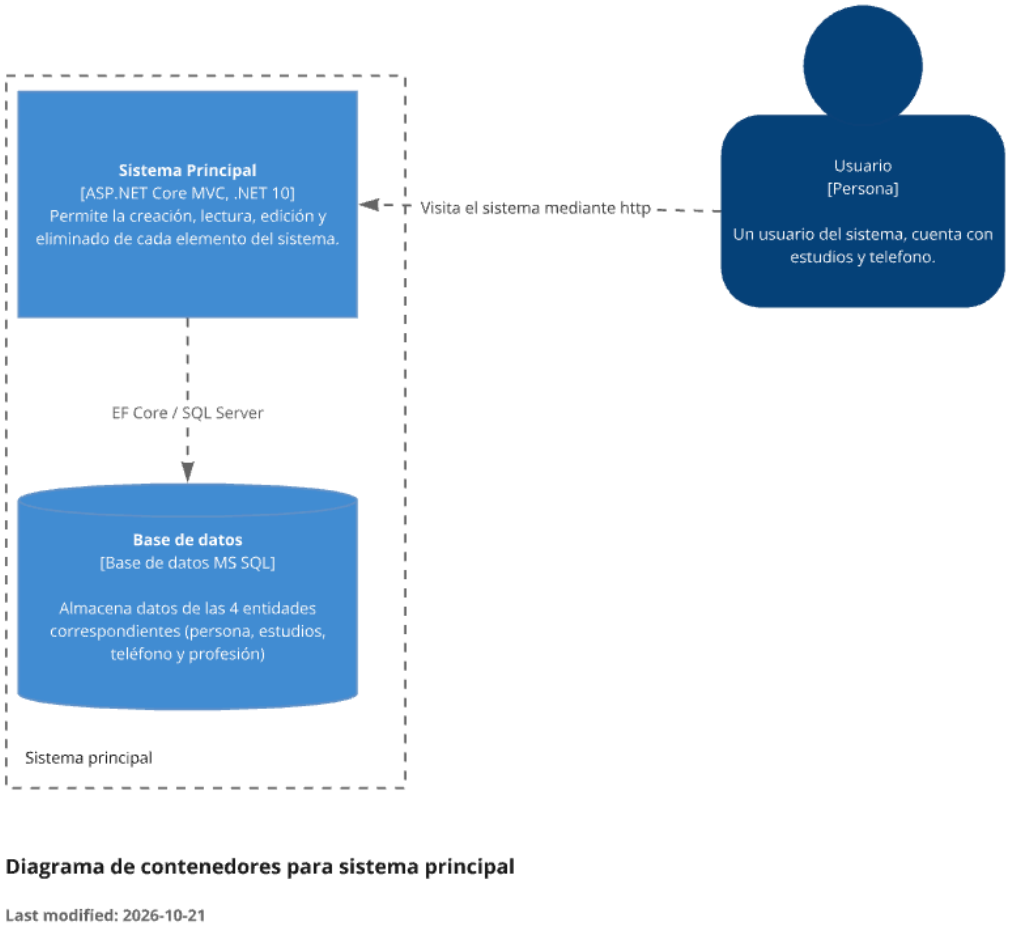


Diagrama 2, Diagrama C4 de contenedores del sistema

El usuario visita el sistema por medio de peticiones HTTP con solicitudes de CRUD para las entidades, estas se tratan según lógica de negocio en el sistema y se pasan a la base de datos en SQL server para su actualización.

El sistema, al seguir una arquitectura monolítica con MVC y DAO, sigue la siguiente arquitectura a nivel de componentes:

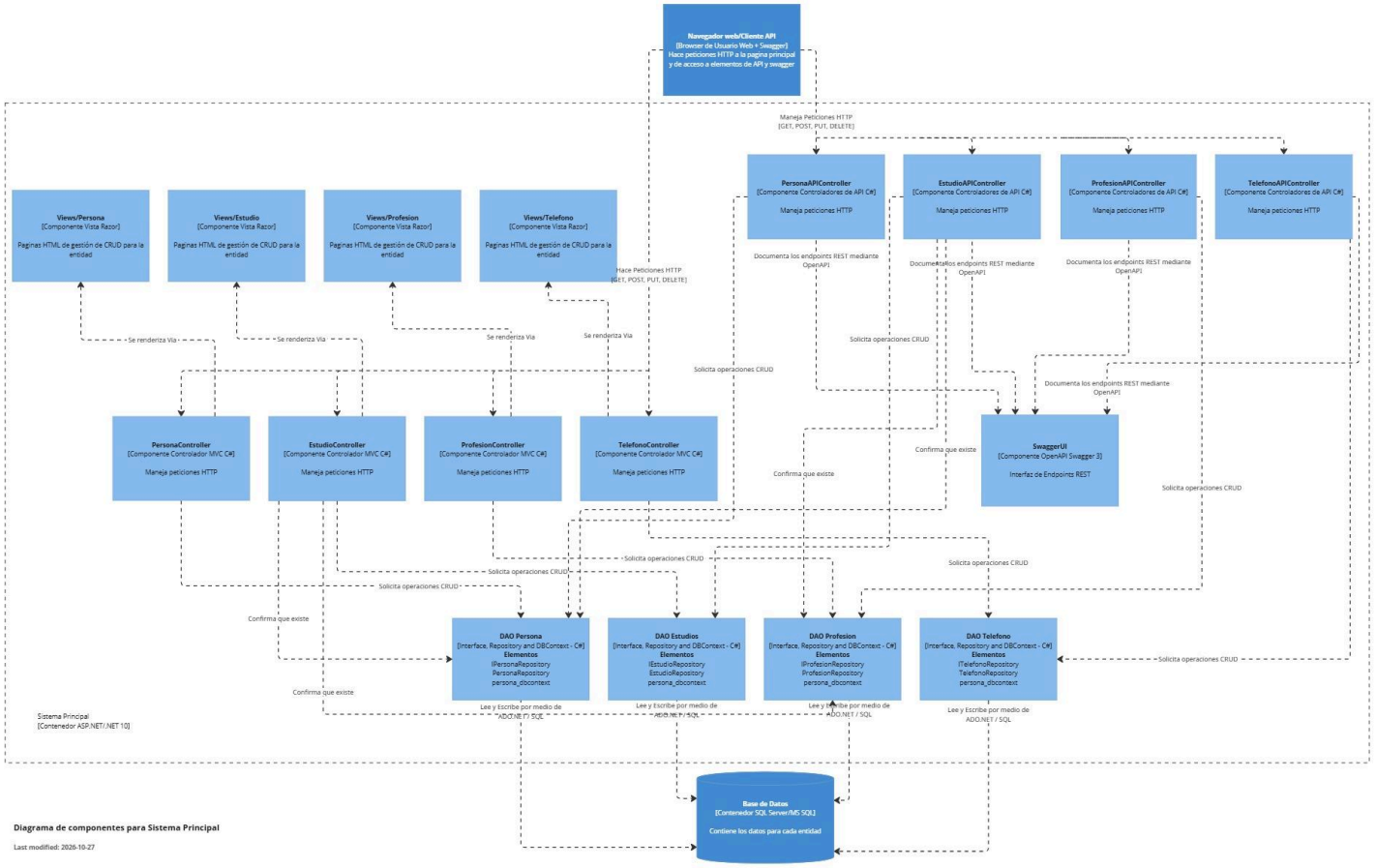


Diagrama 3, Diagrama C4 de componentes del sistema

El usuario interactúa con los controladores MVC y API, encargados de recibir y gestionar las solicitudes correspondientes. Los controladores MVC administran las solicitudes relacionadas con las vistas, y las API Controller exponen los endpoints REST y permiten la interacción con servicios externos. Swagger proporciona la documentación de los endpoints mediante OpenAPI y permite ejecutar solicitudes HTTP de prueba.

Estos controladores utilizan las interfaces IRepository para acceder a las operaciones definidas, empleando los ViewModel cuando corresponde para estructurar y transportar los datos necesarios para las vistas. Las interfaces son implementadas por sus respectivos repositorios, encargados de realizar las operaciones de acceso a datos mediante DbContext y las entidades correspondientes. Finalmente, DbContext gestiona las consultas y modificaciones sobre la base de datos SQL Server (los elementos de DBContext, Repositorio e interfaz se agrupan en su correspondiente DAO). Los resultados obtenidos de las operaciones se utilizan para actualizar las vistas, que posteriormente se presentan al usuario.

A nivel de código, el programa se ve de la siguiente forma:

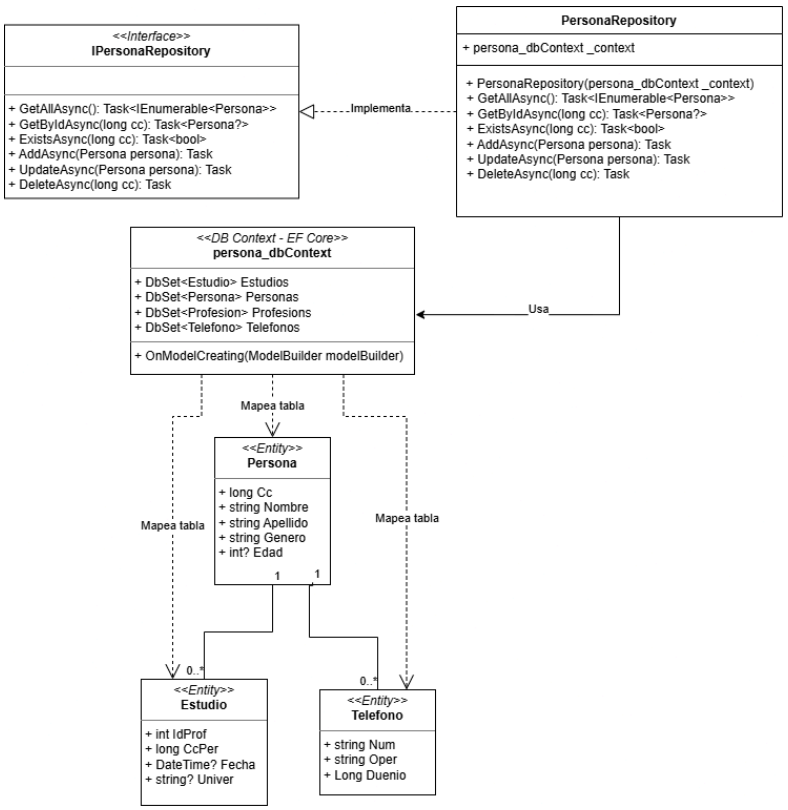


Diagrama 4, Diagrama de código de DAO del sistema

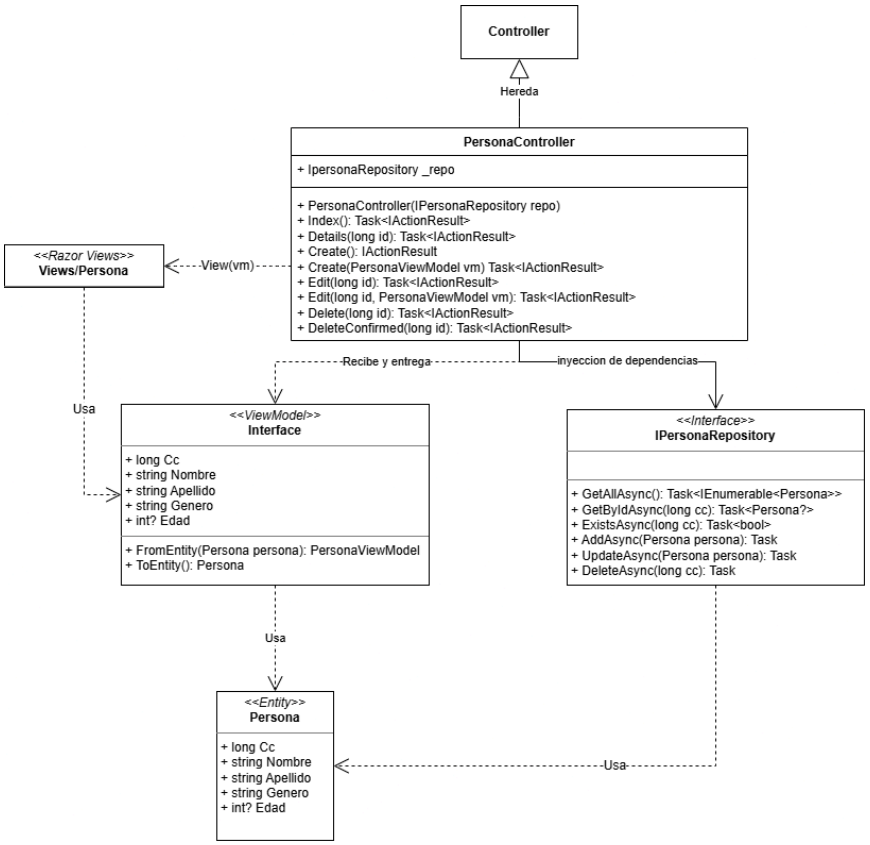


Diagrama 5, Diagrama de código de controladores MVC del sistema

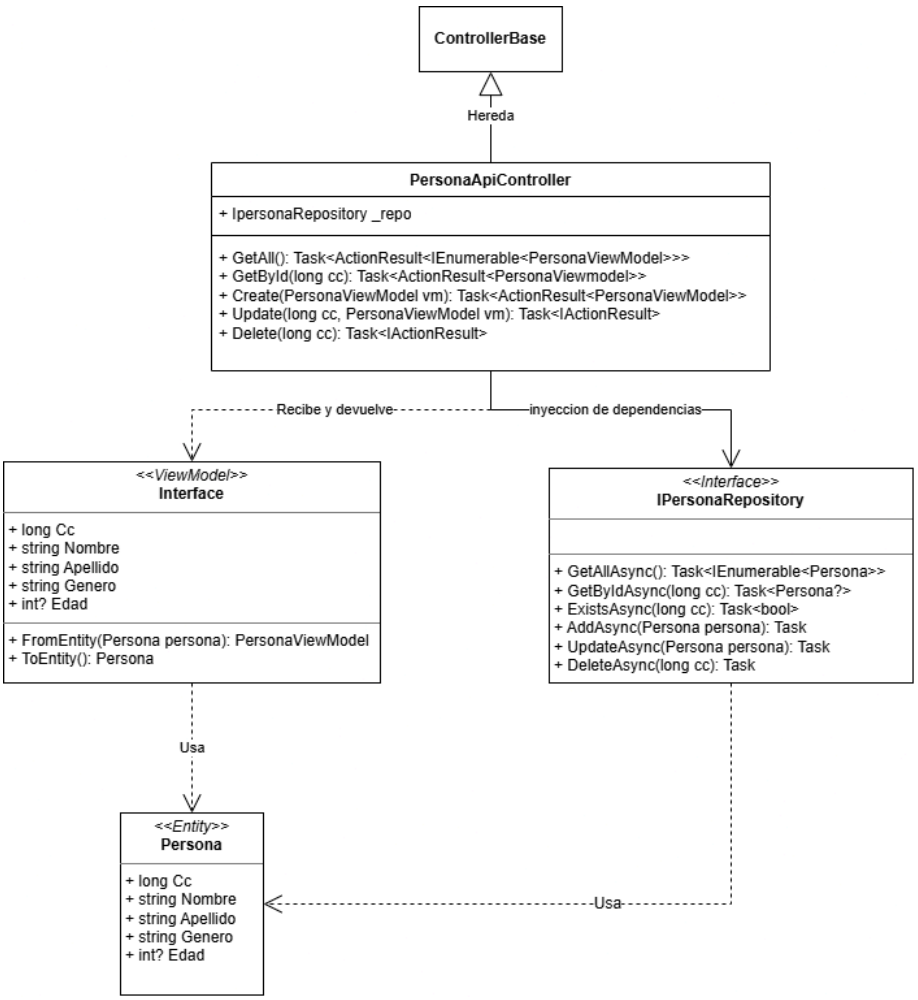


Diagrama 6, Diagrama de código de controladores API del sistema

Los diagramas están representando las interacciones generales entre clases y componentes internos del sistema, representados mediante la entidad Persona. Se utilizan únicamente estos 3 diagramas para esta única entidad dado que todas las demás entidades comparten una lógica similar sino idéntica.

Finalmente, indicando el despliegue del sistema, se cuenta con el siguiente diagrama:

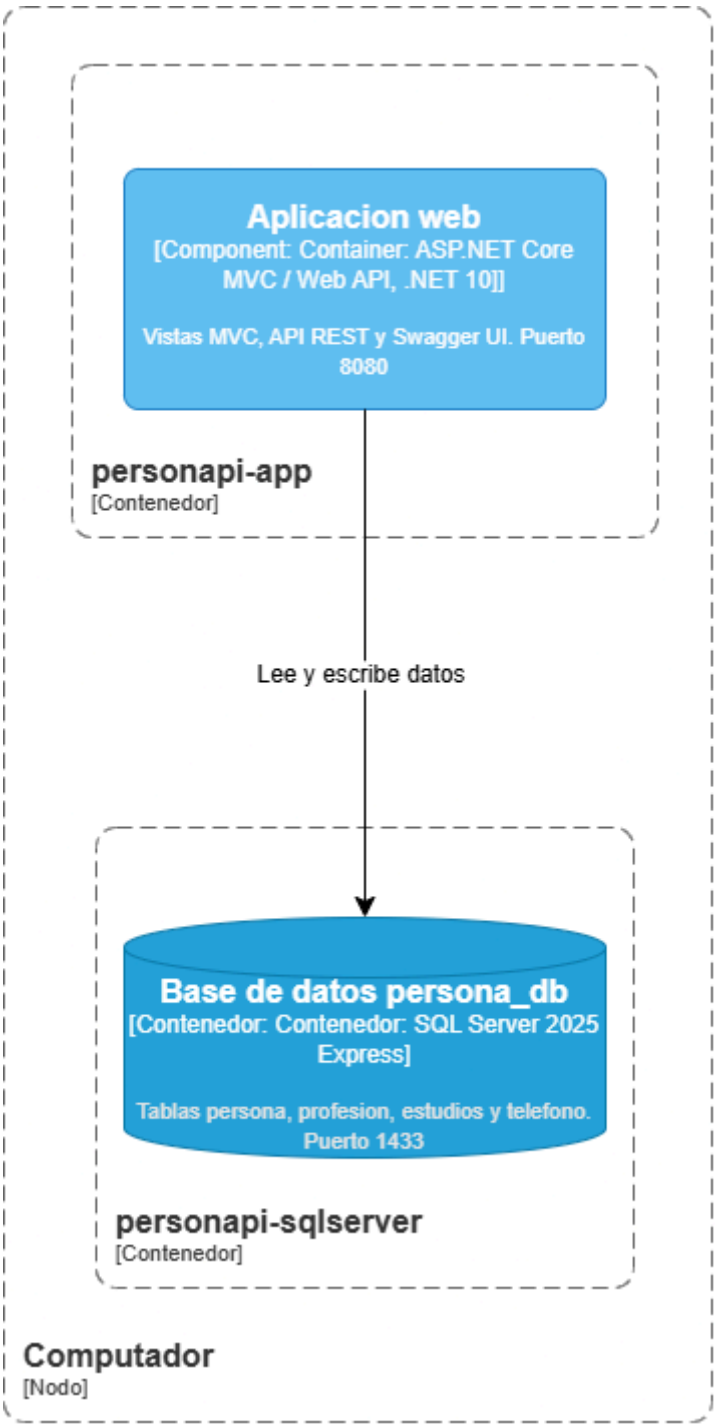


Diagrama 7, Diagrama de despliegue del sistema

Este diagrama muestra la infraestructura del sistema en un entorno local, relacionando la aplicación web con la base de datos dentro del dispositivo local o Computador

Procedimiento

1. Preparación del entorno

El equipo trabajó con versiones más recientes que las del enunciado porque las pedidas ya no son las vigentes; la arquitectura y el código no dependen de esa diferencia.

Componente	**Enunciado**	**Usado**	**Justificación**
Base de datos	SQL Server 2019 Express	SQL Server 2025 Express (instalación Básica)	La 2019 solo recibe soporte extendido; la 2025 es compatible con el mismo T-SQL del laboratorio.
Cliente SQL	SSMS 18	SSMS actual	Se conecta sin problemas a la 2025.
Plataforma	.NET 6 o 7	.NET 10 (net10.0)	.NET 6 y 7 ya no reciben parches de seguridad. Se probó .NET 8, pero Visual Studio 2026 trae .NET 10, que es la versión con la que quedó el proyecto.
IDE	Visual Studio 2022	Visual Studio Community 2026	Es el IDE que instala .NET 10; incluye las cargas ASP.NET y web y Almacenamiento y procesamiento de datos.
Control de versiones	Git + GitHub	Repositorio alex50026/personapi-dot net	Permite trabajo en paralelo por ramas y marcar la entrega con un tag.

Pasos realizados

1. Instalar SQL Server 2025 Express en modo Básico. La instancia queda como **`localhost\SQLEXPRESS`.**
2. Instalar SSMS y conectarse con autenticación de Windows y la casilla **Certificado de servidor de confianza** marcada.
3. Instalar Visual Studio Community 2026 con las dos cargas de trabajo y el runtime de .NET 10.
4. Crear el repositorio en GitHub e invitar a colaboradores.

Por qué se marca "Certificado de servidor de confianza". Las versiones recientes del cliente de SQL Server cifran la conexión por defecto. La instancia local usa un certificado propio que Windows no reconoce, así que sin esa casilla la conexión falla. En el proyecto el mismo ajuste se escribe como **`TrustServerCertificate=true`** en la cadena de conexión (sección 5). Es aceptable en un laboratorio local; en producción se instalaría un certificado válido.

Decisión sobre versiones. Al principio el proyecto se creó con .NET 8. Se migró a .NET 10 al pasar a Visual Studio 2026 para que todo el equipo compile con la misma plataforma. Al subir de versión apareció el error NU1605 (paquetes de Entity Framework en versiones distintas) y avisos de vulnerabilidad NU1902/NU1903. Se resolvieron fijando los tres paquetes de EF Core en la misma versión y añadiendo una referencia directa a **`System.Security.Cryptography.Pkcs`** en una versión corregida.

2. Diseño de la base de datos

La base **`persona_db`** se crea con un script DDL único, **`Scripts/01_ddl.sql`**, que todo el equipo ejecuta en SSMS. Así los tres tienen la misma estructura y se evita que cada uno la cree a mano con diferencias.

```sql
IF DB_ID('persona_db') IS NULL
    CREATE DATABASE persona_db;
GO

USE persona_db;
GO

CREATE TABLE persona (
    cc        BIGINT      NOT NULL PRIMARY KEY,
    nombre    VARCHAR(45) NOT NULL,
    apellido  VARCHAR(45) NOT NULL,
    genero    CHAR(1)     NOT NULL CHECK (genero IN ('M','F')),
    edad      INT         NULL
);

CREATE TABLE profesion (
    id   INT          NOT NULL PRIMARY KEY,
    nom  VARCHAR(90)  NOT NULL,
    des  VARCHAR(MAX) NULL
);
```

```sql
CREATE TABLE estudios (
    id_prof INT         NOT NULL,
    cc_per  BIGINT      NOT NULL,
    fecha   DATE        NULL,
    univer  VARCHAR(50) NULL,
    CONSTRAINT PK_estudios PRIMARY KEY (id_prof, cc_per),
    CONSTRAINT FK_estudios_profesion FOREIGN KEY (id_prof) REFERENCES profesion(id),
    CONSTRAINT FK_estudios_persona   FOREIGN KEY (cc_per)  REFERENCES persona(cc)
);

CREATE TABLE telefono (
    num    VARCHAR(15) NOT NULL PRIMARY KEY,
    oper   VARCHAR(45) NOT NULL,
    duenio BIGINT      NOT NULL,
    CONSTRAINT FK_telefono_persona FOREIGN KEY (duenio) REFERENCES persona(cc)
);
GO
```

Decisiones de tipos

Decisión	Por qué
genero como CHAR(1) con CHECK ('M','F')	El modelo original usaba un ENUM, que SQL Server no tiene. El CHECK da la misma garantía: la base rechaza cualquier otro valor.
cc como BIGINT	Una cédula puede superar el máximo de un INT (2.147.483.647).
num de teléfono como VARCHAR(15)	Un número puede empezar por + o por cero; guardarlo como número perdería esos caracteres.
Claves primarias sin autonumerar	persona.cc, profesion.id y telefono.num los escribe el usuario, así que no se usa IDENTITY. Esto obliga al formulario Create a pedir la clave.
estudios con clave compuesta (id_prof, cc_per)	Es la tabla intermedia de la relación N:M entre persona y profesión; una persona no repite la misma profesión.
Claves foráneas en estudios y telefono	La base misma impide un teléfono sin dueño o un estudio de una persona o profesión inexistentes. Esa regla se refuerza luego en el código (sección 10).

El script no se puede ejecutar dos veces: la segunda vez SQL Server responde que ya existe el objeto **`persona`**. Para repetirlo hay que eliminar antes la base de datos.

3. Creación del proyecto MVC y arquitectura en capas

Se creó una sola solución ASP.NET Core con el patrón MVC y se organizó en carpetas que corresponden a capas, de modo que cada una tenga una única responsabilidad.

Pasos

1. En Visual Studio: **Crear un proyecto**, plantilla **Aplicación web de ASP.NET Core (Modelo-Vista-Controlador)**, nombre **`personapi-dotnet`**, framework .NET 10.
2. Verificar que existan las carpetas **`Controllers`, `Models`** e **Views**, y que **no** exista **`Pages`**.
3. Crear las carpetas propias de la arquitectura: **`Interfaces`, `Repositories`, `Models/Entities`** y **`Models/ViewModels`.**

Carpetas y responsabilidades

Capa	**Carpeta**	**Responsabilidad**
Vista	Views/	Mostrar datos y recoger formularios (Razor). No tiene lógica de acceso a datos.
Controlador MVC	Controllers/<Entidad>Controller.cs	Recibe la petición del navegador, valida y devuelve una vista.
Controlador API	Controllers/<Entidad>ApiController.cs	Expone los mismos datos como REST (JSON) en api/<entidad>.
ViewModel	Models/ViewModels/	Forma de los datos que entran y salen de las vistas y la API, con sus validaciones.

Interfaz DAO	Interfaces/	Contrato de las operaciones de datos (por ejemplo IProfesionRepository).
Repositorio	Repositories/	Implementa el contrato usando Entity Framework.
Entidad y contexto	Models/Entities/	Clases que representan las tablas y el persona_dbContext.
Base de datos	SQL Server persona_db	Almacena los datos.

Decisiones

- **Capas con interfaces.** Los controladores dependen de **`IProfesionRepository`**, no de Entity Framework. Si se cambia la forma de acceder a los datos, los controladores no se tocan, y es posible sustituir el repositorio por uno falso en pruebas.
- **Dos controladores por entidad.** El controlador MVC sirve las pantallas y el de API sirve JSON para Swagger y otros clientes. Comparten el mismo repositorio, así la lógica de datos no se duplica.
- **Un monolito.** Todo se compila y se despliega como una sola aplicación. Para cuatro tablas y un equipo de tres, es lo más simple de desarrollar, probar y entregar, que es el objetivo del laboratorio.
- **Un archivo por responsabilidad y entidad.** Los nombres siguen el patrón **`<Entidad>Controller`**, **`I<Entidad>Repository`**, etc. Eso permitió repartir el trabajo.

4. Capa de modelo: Database First con Scaffold-DbContext

Esta capa responde a una pregunta sencilla: ¿cómo sabe el programa en C# qué tablas y columnas existen en la base de datos? La respuesta es que **no se escribe a mano**: Entity Framework Core lee la estructura de la base **`persona_db`** y genera las clases por nosotros. Este enfoque se llama **Database First** (primero la base de datos, después el código).

Conceptos que se usan en esta sección

- **Entidad.** Es una clase de C# que representa una tabla: cada fila es un objeto de esa clase y cada columna, una propiedad. Por ejemplo, la tabla **`telefono`** se convierte en la clase **`Telefono`**.
- **DbContext.** Es la clase que representa la conexión con la base de datos y agrupa las tablas como colecciones (**`DbSet`**). Aquí se llama **`persona_dbContext`**, y los repositorios la usan para consultar y guardar datos (por ejemplo **`_context.Telefonos`**).
- **Scaffold-DbContext.** Es un comando de Entity Framework Core que se conecta a la base, lee su estructura (tablas, columnas, tipos, claves primarias y claves foráneas) y escribe los archivos de las entidades y del DbContext.

De dónde sale la información

Todo parte del script **`Scripts/01_ddl.sql`** (sección 2), que crea las tablas en SQL Server. El comando no lee ese archivo: se conecta a la base ya creada, consulta su estructura y la traduce a código. El recorrido completo es:

```
Scripts/01_ddl.sql  ->  base persona_db en SQL Server  ->  Scaffold-DbContext  ->  carpeta Models/Entities
```

Pasos

1. Instalar desde **NuGet** (clic derecho en el proyecto, **Administrar paquetes NuGet**) los tres paquetes de Entity Framework Core, todos en la misma versión: **`Microsoft.EntityFrameworkCore`**, **`.SqlServer`** y **`.Tools`**. El paquete **`.Tools`** es el que habilita el comando.
2. Abrir la **Consola del Administrador de paquetes** (menú **Herramientas > Administrador de paquetes NuGet**) y ejecutar el comando de abajo.
3. Verificar que en **`Models/Entities`** aparezcan **`Estudio`**, **`Persona`**, **`Profesion`**, **`Telefono`** y **`persona_dbContext`**: una clase por cada tabla y el contexto.
4. Quitar de **`persona_dbContext`** el método **`OnConfiguring`**, que trae la cadena de conexión escrita en el código (sección 5).

```
Scaffold-DbContext "Server=localhost\SQLEXPRESS;Database=persona_db;Trusted_Connection=True;TrustServerCertificate=true" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models/Entities
```

El comando, parte por parte

Parte del comando	Qué significa
`Server=localhost\SQLEXPRESS`	Instancia de SQL Server donde está la base de datos.
`Database=persona_db`	Base de datos que se va a leer.

`Trusted_Connection=True`	Se conecta con la sesión de Windows; no hay usuario ni contraseña en el comando.
`TrustServerCertificate=true`	Acepta el certificado propio de la instancia local (sección 1).
`Microsoft.EntityFrameworkCore.SqlServer`	Proveedor: indica a Entity Framework Core que la base es SQL Server.
`-OutputDir Models/Entities`	Carpeta donde se escriben los archivos generados.

Con una sola ejecución se generan los archivos de las cuatro tablas (**`persona`**, **`profesion`**, **`estudios`** y **`telefono`**) y el contexto. A continuación se muestra qué produce el comando con la tabla **`telefono`**, que aparece en tres lugares.

1. En la base de datos (Scripts/01_ddl.sql)

```
CREATE TABLE telefono (
	num    VARCHAR(15) NOT NULL,
	oper   VARCHAR(45) NOT NULL,
	duenio BIGINT       NOT NULL,
	CONSTRAINT PK_telefono           PRIMARY KEY (num),
	CONSTRAINT FK_telefono_persona FOREIGN KEY (duenio) REFERENCES persona(cc)
);
```

2. La entidad generada (Models/Entities/Telefono.cs)

Cada columna pasó a ser una propiedad. **`DuenioNavigation`** es la propiedad que apunta a la persona dueña del teléfono; se crea a partir de la clave foránea.

```
public partial class Telefono
{
	public string Num { get; set; } = null!;
	public string Oper { get; set; } = null!;
	public long Duenio { get; set; }

	public virtual Persona DuenioNavigation { get; set; } = null!;
}
```

3. El contexto generado (Models/Entities/persona_dbContext.cs)

El contexto declara la tabla como una colección y, en **`OnModelCreating`**, describe cómo se corresponde cada propiedad con cada columna (fragmento resumido):

```
public virtual DbSet<Telefono> Telefonos { get; set; } = null!;

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
```

```
        modelBuilder.Entity<Telefono>(entity =>
        {
        entity.HasKey(e => e.Num);
        entity.ToTable("telefono");

        entity.Property(e => e.Num)
               .HasMaxLength(15)
               .IsUnicode(false)
               .HasColumnName("num");

        entity.Property(e => e.Duenio).HasColumnName("duenio");

        entity.Property(e => e.Oper)
               .HasMaxLength(45)
               .IsUnicode(false)
               .HasColumnName("oper");

        entity.HasOne(d => d.DuenioNavigation)
               .WithMany(p => p.Telefonos)
               .HasForeignKey(d => d.Duenio)
              .OnDelete(DeleteBehavior.ClientSetNull)
              .HasConstraintName("FK_telefono_persona");
        });
        ...
}
```

Cómo se corresponden la base de datos y el código

En la base de datos (SQL)	**En el código generado (C#)**
Tabla `telefono`	Clase `Telefono` y colección `DbSet<Telefono> Telefonos`; `ToTable("telefono")` las une.
Columna `num VARCHAR(15)`	Propiedad `string Num`. `HasMaxLength(15)` conserva el límite y `HasColumnName("num")` la une con la columna (en C# los nombres empiezan en mayúscula).
`VARCHAR` (no Unicode)	`IsUnicode(false)`
Columna `duenio BIGINT NOT NULL`	`long Duenio`, sin `?` porque la columna no admite nulos.
Columna que sí admite nulos, como `des VARCHAR(MAX) NULL` en `profesion`	`string? Des`: el `?` indica que el valor puede ser nulo.
`PRIMARY KEY (num)`	`HasKey(e => e.Num)`

`FOREIGN KEY (duenio) REFERENCES persona(cc)`	`HasOne ... WithMany ... HasForeignKey` y la propiedad de navegación `DuenioNavigation`.

`HasOne ... WithMany` expresa la relación 1:N del modelo de datos: un teléfono tiene un dueño (**`DuenioNavigation`**) y una persona tiene muchos teléfonos (**`Telefonos`**). Gracias a eso se puede llegar desde un teléfono a su dueño sin escribir SQL (con **`Include`**, sección 6).

Las demás tablas se generan con el mismo procedimiento; solo cambian las columnas y las relaciones. En **`estudios`**, por ejemplo, la clave primaria es compuesta (**`HasKey(e => new { e.IdProf, e.CcPer })`**) y tiene dos claves foráneas.

Decisiones

- **Database First y no Code First.** El modelo de datos ya estaba definido y la base ya existía; generar las clases desde ella garantiza que código y tablas no se desvíen, y ahorra escribirlas a mano.
- **Nunca usar `-Force`.** Ese parámetro sobrescribe todas las entidades y volvería a añadir **`OnConfiguring`**. Como las entidades quedan compartidas por el equipo, se generaron una sola vez y se subieron al repositorio.
- **Las entidades no se editan.** Representan las tablas y nada más. Las validaciones y los campos de pantalla van en los ViewModels (sección 7), así una regeneración futura no borra trabajo del equipo.

5. Configuración: conexión, inyección de dependencias y Swagger

Todo el cableado de la aplicación está en dos archivos: **`appsettings.json`**, que guarda la cadena de conexión, y **`Program.cs`**, que registra los servicios.

Cadena de conexión (`appsettings.json`)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost\\SQLEXPRESS;Database=persona_db;Trusted_Connection=True;TrustServerCertificate=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

`Trusted_Connection=True` usa la sesión de Windows, así no hay contraseña en el repositorio. Cada integrante cambia el nombre de su instancia solo en su equipo si es distinto.

Registro de servicios (`Program.cs`)

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews(options =>
        options.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes = true)
    .AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.Ignor
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<persona_dbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Línea	**Qué hace**
AddControllersWithViews()	Activa controladores MVC con vistas Razor y también los controladores de API.
ReferenceHandler.IgnoreCycles	Persona tiene teléfonos y cada teléfono apunta a su persona; al convertir a JSON eso formaría un ciclo infinito. Esta opción lo corta como protección.
AddSwaggerGen() y UseSwagger()	Generan la especificación OpenAPI y la página /swagger para probar los endpoints. Se usó Swashbuckle.AspNetCore.
AddDbContext<persona_dbContext>	Registra el contexto de EF Core con la cadena de conexión leída de la configuración.
AddScoped<IProfesionRepository, ProfesionRepository>()	Le dice al contenedor: cuando un controlador pida la interfaz, entrégale esta clase.
MapControllerRoute	Ruta por convención de las pantallas MVC: /Profesion/Edit/3. Las rutas api/... se declaran con atributos en cada controlador de API.

Decisiones

- **Inyección de dependencias.** Los controladores reciben su repositorio por el constructor y nunca lo crean con **`new`**. Es lo que hace posible cambiar la implementación sin tocar los controladores. Si falta la línea **`AddScoped`** de una entidad, la aplicación falla con "Unable to resolve service for type 'I...Repository'".
- **Ciclo de vida Scoped.** Se crea un repositorio y un contexto por petición HTTP y se descartan al terminar. Evita compartir un contexto entre peticiones y mantiene los datos consistentes.
- **La cadena de conexión sale del código.** Por eso se eliminó **`OnConfiguring`** de las entidades generadas: dejar la cadena allí sería tenerla escrita en dos lugares y es una mala práctica de seguridad.

6. Capa de acceso a datos: patrón DAO con repositorios

Cada entidad tiene una interfaz que dice qué operaciones existen y una clase que las implementa con Entity Framework. Los controladores solo conocen la interfaz.

Pasos por entidad

1. Crear **`Interfaces/I<Entidad>Repository.cs`** con las operaciones CRUD.
2. Crear **`Repositories/<Entidad>Repository.cs`** que reciba **`persona_dbContext`** por constructor e implemente la interfaz.
3. Registrar el par en **`Program.cs`** con **`AddScoped`**.

Interfaz de Profesion

```
10 referencias
public interface IProfesionRepository
{
    3 referencias
    Task<List<Profesion>> GetAllAsync();
    8 referencias
    Task<Profesion?> GetByIdAsync(int id);
    3 referencias
    Task AddAsync(Profesion profesion);
    3 referencias
    Task UpdateAsync(Profesion profesion);
    3 referencias
    Task DeleteAsync(int id);
    6 referencias
    Task<bool> ExistsAsync(int id);
    3 referencias
    Task<bool> HasEstudiosAsync(int id);

}
```

Son las cuatro operaciones CRUD más dos consultas de apoyo: **`ExistsAsync`** (para validar antes de crear o editar) y **`HasEstudiosAsync`** (para la regla de borrado de la sección 10).

Implementación de Profesion

```
2 referencias
public class ProfesionRepository : IProfesionRepository
{
    12 referencias
    private readonly persona_dbContext _context;

    0 referencias
    public ProfesionRepository(persona_dbContext context)
    {
        _context = context;
    }

    3 referencias
    public async Task<List<Profesion>> GetAllAsync() =>
        await _context.Profesions.AsNoTracking().OrderBy(p => p.Id).ToListAsync();

    8 referencias
    public async Task<Profesion?> GetByIdAsync(int id) =>
        await _context.Profesions.AsNoTracking().FirstOrDefaultAsync(p => p.Id == id);
```

```
3 referencias
public async Task AddAsync(Profesion profesion)
{
    _context.Profesions.Add(profesion);
    await _context.SaveChangesAsync();
}

3 referencias
public async Task UpdateAsync(Profesion profesion)
{
    _context.Profesions.Update(profesion);
    await _context.SaveChangesAsync();
}

3 referencias
public async Task DeleteAsync(int id)
{
    var entity = await _context.Profesions.FindAsync(id);
    if (entity is null) return;
    _context.Profesions.Remove(entity);
    await _context.SaveChangesAsync();
}
```

```
        3 referencias
        public async Task DeleteAsync(int id)
        {
            var entity = await _context.Profesions.FindAsync(id);
            if (entity is null) return;
            _context.Profesions.Remove(entity);
            await _context.SaveChangesAsync();
        }

        6 referencias
        public async Task<bool> ExistsAsync(int id) =>
            await _context.Profesions.AnyAsync(p => p.Id == id);

        3 referencias
        public async Task<bool> HasEstudiosAsync(int id) =>
            await _context.Estudios.AnyAsync(e => e.IdProf == id);
    }
}
```

Variante con relación: Telefono. Su repositorio sigue el mismo molde, pero con la clave de tipo **`string`** y cargando al dueño para poder mostrar su nombre:

```
3 referencias
public async Task<List<Telefono>> GetAllAsync() =>
    await _context.Telefonos
        .AsNoTracking()
        .Include(t => t.DuenioNavigation)
        .OrderBy(t => t.Num)
        .ToListAsync();
```

Persona y Estudio. Siguen el mismo molde de interfaz y repositorio; solo cambian las columnas y las reglas propias de cada entidad. La diferencia más importante es el borrado de **`Persona`**, que elimina también sus teléfonos y estudios dentro de una sola transacción (sección 10).

Decisiones

- **Interfaz más implementación.** El controlador pide **`IProfesionRepository`**; quién la cumple lo decide **`Program.cs`**. Esto es el patrón DAO: la persistencia queda encapsulada y reemplazable.
- **Métodos asíncronos.** Cada acceso a la base usa **`async/await`**, para que el servidor no bloquee un hilo mientras SQL Server responde.
- **`AsNoTracking()` en las lecturas.** Las consultas de solo lectura no necesitan que EF siga los cambios de los objetos; es más rápido y evita conflictos al actualizar después.
- **SaveChangesAsync dentro del repositorio.** Cada operación queda completa en una llamada. En Persona, las tres eliminaciones se confirman juntas: si una falla, ninguna se aplica.
- **`Include` solo donde se necesita. `Telefono`** carga al dueño porque las pantallas muestran su nombre; **`Profesion`** no carga estudios porque las pantallas no los muestran.
- **El repositorio devuelve entidades, no ViewModels.** Conoce la base, no la pantalla; la conversión a ViewModel la hace el controlador (sección 7).

7. ViewModels: no exponer las entidades

Las pantallas y la API no reciben ni devuelven las entidades de Entity Framework, sino un ViewModel por entidad, ubicado en **`Models/ViewModels`**. El ViewModel decide qué campos viajan y cómo se validan.

Por qué se hizo. Con la entidad **`Telefono`** directamente en la API pasó lo siguiente: al crear un teléfono desde Swagger, el cuerpo incluía el campo **`duenioNavigation`** (el objeto Persona completo) y la petición se rechazaba con error 400, sin que el mensaje explicara la causa. La entidad mezcla datos de la tabla con navegaciones a otras entidades; el ViewModel deja solo lo que el usuario debe enviar.

ViewModel de Profesion (`Models/ViewModels/ProfesionViewModel.cs`)

```
namespace personapi_dotnet.Models.ViewModels
{
    28 referencias
    public class ProfesionViewModel
    {
        [Required(ErrorMessage = "El Id es obligatorio.")]
        [Range(1, int.MaxValue, ErrorMessage = "El Id debe ser un número mayor que 0.")]
        26 referencias
        public int? Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio.")]
        [StringLength(90, ErrorMessage = "El nombre admite máximo 90 caracteres.")]
        [Display(Name = "Nombre")]
        14 referencias
        public string Nom { get; set; } = string.Empty;

        [Display(Name = "Descripción")]
        15 referencias
        public string? Des { get; set; }
```

```
        8 referencias
        public static ProfesionViewModel FromEntity(Profesion p) => new()
        {
            Id = p.Id,
            Nom = p.Nom,
            Des = p.Des
        };

        4 referencias
        public Profesion ToEntity() => new()
        {
            Id = Id!.Value,
            Nom = Nom.Trim(),
            Des = string.IsNullOrWhiteSpace(Des) ? null : Des.Trim()
        };
    }
}
```

ViewModel de Telefono, con los campos adicionales que solo sirven a las pantallas:

```
[Required(ErrorMessage = "Seleccione un dueño.")]
[Range(1, long.MaxValue, ErrorMessage = "La cédula del dueño debe ser mayor que 0.")]
[Display(Name = "Dueño (cédula)")]
25 referencias
public long? Duenio { get; set; }

// Solo para mostrar en Index/Details/Delete.
[ValidateNever]
[JsonIgnore]
[Display(Name = "Dueño")]
7 referencias
public string? DuenioNombre { get; set; }

// Solo para el desplegable de las vistas Create/Edit. No viaja en la API.
[ValidateNever]
[JsonIgnore]
3 referencias
public IEnumerable<SelectListItem>? Personas { get; set; }
```

Elemento	Función
`[Required]`, **`[StringLength]`**, **`[Range]`**, **`[RegularExpression]`**	Reglas de validación declaradas una vez; se aplican en el formulario y en la API. Los límites (45, 90, 15 caracteres) copian los de la base.
`[Display(Name = ...)]`	Texto de la etiqueta en las pantallas.
`FromEntity` / **`ToEntity`**	Convierten entre la entidad de la base y el ViewModel, en un solo lugar.
`[ValidateNever]` y **`[JsonIgnore]`**	**`DuenioNombre`** y **`Personas`** existen solo para dibujar la pantalla: no se validan y no aparecen en el JSON de la API.
`int?` y **`long?`** en las claves	Si el campo llega vacío, el usuario ve "El Id es obligatorio" en vez de un error técnico de conversión.

Decisiones

- **La entidad queda como modelo de la tabla y nada más.** Regenerarla desde la base no borra reglas de validación, porque estas viven en el ViewModel.

- **Una sola definición de las reglas.** El mismo ViewModel sirve a las vistas y a la API, así los dos caminos validan igual.
- **Mismo estilo para todas las entidades.** Los ViewModels de **`Persona`**, **`Estudio`**, **`Profesion`** y **`Telefono`** siguen el mismo esquema (**`FromEntity`**/**`ToEntity`** y atributos de validación), así que los controladores los usan de la misma forma.

8. Controladores MVC y API REST

Cada entidad tiene dos controladores que usan el mismo repositorio: uno devuelve pantallas y otro devuelve JSON. Los dos siguen la misma secuencia: validar, consultar al repositorio, responder.

Controlador MVC de Profesion (fragmento)

```
1 referencia
public class ProfesionController : Controller
{
    12 referencias
    private readonly IProfesionRepository _repo;

    0 referencias
    public ProfesionController(IProfesionRepository repo)
    {
        _repo = repo;
    }

    3 referencias
    public async Task<IActionResult> Index()
    {
        var lista = await _repo.GetAllAsync();
        return View(lista.Select(ProfesionViewModel.FromEntity));
    }
```

```
[HttpPost]
[ValidateAntiForgeryToken]
0 referencias
public async Task<IActionResult> Create(ProfesionViewModel vm)
{
    if (vm.Id.HasValue && await _repo.ExistsAsync(vm.Id.Value))
        ModelState.AddModelError(nameof(vm.Id), "Ya existe una profesión con ese Id.");

    if (!ModelState.IsValid) return View(vm);

    try
    {
        await _repo.AddAsync(vm.ToEntity());
    }
    catch (DbUpdateException)
    {
        ModelState.AddModelError(string.Empty, "No se pudo guardar la profesión. Intente de nuevo.");
        return View(vm);
    }
    return RedirectToAction(nameof(Index));
}
```

El constructor recibe la interfaz, no la clase. En **`Create`**, primero se añade el error de "Id repetido" al estado del modelo; si hay cualquier error (el de las anotaciones del ViewModel o este), se vuelve a mostrar el formulario con los mensajes; si no, se guarda y se redirige a la lista. **`[ValidateAntiForgeryToken]`** rechaza envíos de formularios que no vengan de la propia aplicación.

Controlador de API de Profesion (fragmento)

```
[ApiController]
[Route("api/profesion")]
1 referencia
public class ProfesionApiController : ControllerBase
{
    10 referencias
    private readonly IProfesionRepository _repo;

    0 referencias
    public ProfesionApiController(IProfesionRepository repo)
    {
        _repo = repo;
    }

    [HttpGet]
    0 referencias
    public async Task<ActionResult<IEnumerable<ProfesionViewModel>>> GetAll()
    {
        var lista = await _repo.GetAllAsync();
        return Ok(lista.Select(ProfesionViewModel.FromEntity));
    }
```

```
[HttpGet("{id:int}")]
1 referencia
public async Task<ActionResult<ProfesionViewModel>> GetById(int id)
{
    var p = await _repo.GetByIdAsync(id);
    if (p is null) return NotFound();
    return Ok(ProfesionViewModel.FromEntity(p));
}

[HttpPost]
0 referencias
public async Task<ActionResult<ProfesionViewModel>> Create(ProfesionViewModel vm)
{
    if (await _repo.ExistsAsync(vm.Id!.Value))
        return Conflict(new { message = "Ya existe una profesión con ese Id." });

    try
    {
        await _repo.AddAsync(vm.ToEntity());
    }
    catch (DbUpdateException)
    {
        return Conflict(new { message = "No se pudo guardar la profesión." });
    }
    return CreatedAtAction(nameof(GetById), new { id = vm.Id }, vm);
```

`[ApiController]` valida el ViewModel solo y responde 400 si falla, sin código adicional. **`{id:int}`** en la ruta restringe el parámetro a enteros: **`/api/profesion/abc`** no llega al método.

Códigos HTTP usados

Operación	Verbo y ruta	Éxito	Errores
Listar	**`GET /api/profesion`**	200	–
Consultar uno	**`GET /api/profesion/{id}`**	200	404 si no existe

Crear	**`POST /api/profesion`**	201 con la ruta del nuevo recurso	400 datos inválidos, 409 si el Id ya existe
Actualizar	**`PUT /api/profesion/{id}`**	204	400 si el Id de la ruta no coincide con el del cuerpo, 404 si no existe
Eliminar	**`DELETE /api/profesion/{id}`**	204	404 si no existe, 409 si tiene estudios asociados

En Telefono, además, crear o actualizar con un dueño inexistente responde 400 con el detalle del campo **`Duenio`**; se comprueba con **`IPersonaRepository.ExistsAsync`**.

Decisiones

- **Controladores delgados.** No contienen consultas ni SQL: validan, llaman al repositorio y eligen la respuesta.
- **Mensajes claros en vez de errores internos.** Antes de las correcciones, un **PUT** sobre un teléfono inexistente respondía con un 400 engañoso. Ahora la API distingue 404 (no existe), 409 (conflicto con el estado actual) y 400 (datos inválidos).
- **El mismo repositorio sirve a los dos controladores.** La lógica de datos existe una sola vez.

9. Vistas Razor

Cada entidad tiene cinco vistas en **`Views/<Entidad>/`**: Index (lista), Details, Create, Edit y Delete. Se escriben en Razor, que mezcla HTML con código C#, y reciben un ViewModel.

Pasos

1. Crear las acciones del controlador MVC.
2. Clic derecho en cada acción, **Agregar vista**, **Vista de Razor**, y la plantilla del mismo nombre (Index, Details, Create, Edit o Delete).
3. Cambiar el modelo de cada vista por el ViewModel y borrar los campos que la plantilla añade por defecto para las colecciones relacionadas (por ejemplo **`Estudios`** o **`Telefonos`**).

4. Comprobar en el navegador cada pantalla.

El generador de vistas falló con el error "No properties found for --model" porque el proyecto no estaba compilado; las vistas de Telefono se escribieron a mano con la misma estructura. Compilar antes de generar evita ese error.

Formulario Create de Telefono (fragmento)

```
@model TelefonoViewModel

<form asp-action="Create">
     <div asp-validation-summary="ModelOnly"
class="text-danger"></div>

     <div class="mb-3">
     <label asp-for="Num" class="form-label"></label>
     <input asp-for="Num" class="form-control" />
     <span asp-validation-for="Num" class="text-danger"></span>
     </div>

     <div class="mb-3">
     <label asp-for="Duenio" class="form-label"></label>
     <select asp-for="Duenio" asp-items="Model.Personas"
class="form-select">
          <option value="">-- Seleccione una persona --</option>
     </select>
     <span asp-validation-for="Duenio" class="text-danger"></span>
     </div>

     <button type="submit" class="btn btn-primary">Guardar</button>
</form>

@section Scripts {
     @{ await Html.RenderPartialAsync("_ValidationScriptsPartial");
}
}
```

Etiqueta	Función
`@model TelefonoViewModel`	Declara qué objeto recibe la vista; da autocompletado y validación en compilación.

`asp-action`, **`asp-for`**	Generan la URL y el campo del formulario a partir del controlador y del ViewModel.
`asp-validation-for`	Muestra el mensaje de error de ese campo, el mismo de las anotaciones del ViewModel.
`asp-items="Model.Personas"`	Llena el desplegable con la lista de personas que prepara el controlador.
`_ValidationScriptsPartial`	Valida en el navegador antes de enviar, usando las mismas reglas.

Decisiones

- **Clave editable solo al crear.** Como las claves no se autogeneran, Create las pide; en Edit el campo clave es de solo lectura, porque cambiarla equivaldría a crear otro registro.
- **Desplegable para el dueño.** Escribir a mano la cédula del dueño permitía errores; el desplegable limita la elección a personas que existen.
- **Mismo estilo para las cinco vistas de cada entidad.** Index lista, Details y Delete muestran los datos en una lista de definición, y Create y Edit comparten los mismos campos.

10. Integridad y manejo de errores

Los datos se protegen en tres niveles. Cada nivel detecta un tipo distinto de error y da un mensaje que el usuario puede entender.

Nivel	Qué verifica	Ejemplo	Respuesta al usuario
ViewModel (anotaciones)	Formato y longitud de cada campo	Nombre vacío o de más de 90 caracteres	Mensaje bajo el campo; 400 en la API

Controlador (consulta al repositorio)	Reglas que dependen de otros datos	Id repetido; dueño inexistente; profesión con estudios	Mensaje en el formulario; 409 o 400 en la API
Base de datos (PK, FK, CHECK)	Última defensa, aunque el código falle	Clave duplicada, clave foránea sin padre	**`DbUpdateException`**, capturada y convertida en mensaje

Captura de errores de la base

```
try
{
      await _repo.AddAsync(vm.ToEntity());
}
catch (DbUpdateException)
{
    ModelState.AddModelError(string.Empty, "No se pudo guardar la profesión. Intente de nuevo.");
      return View(vm);
}
```

Sin este bloque, una violación de clave de SQL Server aparecía como una pantalla de error. Con él, la persona vuelve al formulario con un mensaje, y los datos que escribió siguen ahí.

Política de borrado por entidad

Entidad	Qué pasa al eliminar	Por qué
Persona	Se eliminan también sus teléfonos y sus estudios, en una sola transacción	Esos registros no tienen sentido sin la persona: dependen de ella.
Telefono	Se elimina directamente	No tiene registros que dependan de él.

Profesion	Se bloquea si tiene estudios asociados: la pantalla muestra un mensaje y la API responde 409	Una profesión es un dato compartido; borrarla en cascada eliminaría estudios de personas que no se estaban modificando.

Implementación del bloqueo en Profesion

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
      var p = await _repo.GetByIdAsync(id);
      if (p is null) return NotFound();

      if (await _repo.HasEstudiosAsync(id))
      {
         ModelState.AddModelError(string.Empty,
            "No se puede eliminar: hay estudios asociados a esta
profesión.");
      return View("Delete", ProfesionViewModel.FromEntity(p));
      }

      await _repo.DeleteAsync(id);
      return RedirectToAction(nameof(Index));
}
```

Decisiones

- **La regla de borrado se decide por entidad, no una para todas.** La pregunta es quién depende de quién: lo que depende de una persona se va con ella; lo que otros usan, se protege.
- **Validar en el código aunque la base ya tenga restricciones.** La base impide el dato incorrecto, pero su mensaje es técnico. Validar antes permite explicar qué pasó.
- **La respuesta de error es parte del diseño.** 409 significa "la petición es válida pero choca con el estado actual", y es distinto de 400 (petición mal formada) y 404 (no existe).

11. Trabajo en equipo con Git

El repositorio **`personapi-dotnet`** concentró el trabajo de los tres integrantes. La base común (proyecto, entidades, script DDL y configuración) se subió primero a **`main`**; luego cada uno construyó sus entidades en archivos propios.

Flujo seguido

1. Un integrante creó el proyecto, las entidades y el script **`Scripts/01_ddl.sql`**, y lo subió a **`main`** con **Confirmar todo e insertar**.
2. Los demás integrantes clonaron el repositorio y trabajaron en ramas propias, una por entidad (por ejemplo **`feat/EstudiosCorrected`** y **`feat/Persona`**), además de una rama **`develop`** de integración.
3. Cada cambio se integró con pull request o con una combinación de ramas en GitHub (hubo PR para **`feat/Persona`** y para **`feat/EstudiosCorrected`**).
4. Al final se combinó **`develop`** en **`main`**.

Reglas acordadas para no pisarse

- Hacer **Extraer** (pull) antes de empezar a trabajar.
- Editar solo los archivos de la propia entidad. Los archivos compartidos (**`Program.cs`**, **`_Layout.cshtml`**, **`.csproj`**) se tocan lo mínimo.
- En **`Program.cs`**, cada integrante activa solo la línea **`AddScoped`** de su entidad.
- No ejecutar **`Scaffold-DbContext`** con **`-Force`**.
- Hacer commits pequeños y frecuentes con mensajes que digan qué cambió.
- Si Git avisa de un conflicto (casi siempre en **`Program.cs`** o en el **`.csproj`**), conservar las líneas de ambos lados y avisar al equipo.

Decisiones

- **Una rama por entidad y una de integración.** El código a medias de una persona no rompe la compilación de las otras dos, y **`main`** solo recibe lo que ya compila.
- **Archivos separados por entidad.** Es lo que hizo que los conflictos fueran pocos: las entidades distintas casi no comparten archivos; los conflictos aparecieron en **`Program.cs`**, que todos modifican.
- **Un `.csproj` común con versiones fijas.** Al quedar los tres paquetes de EF Core en la misma versión y sin la referencia a **`Microsoft.Data.SqlClient`**, los tres compilan igual.

Entrega. La versión entregable se marca con un tag sobre **`main`** (por ejemplo **`v1.0`**), creado desde Visual Studio con **el menú Git** y subido al repositorio. Queda pendiente hasta que las cuatro entidades estén integradas y probadas.

12. Pruebas, despliegue y resumen de decisiones

La aplicación se prueba por dos caminos que ejercitan el mismo código: las pantallas del navegador (**`/Profesion`**, **`/Telefono`**) y Swagger (**`/swagger`**). Cada caso se registra con una captura para el informe.

Casos de prueba de Profesion y Telefono

Caso	Resultado esperado en pantalla	Resultado esperado en la API
Crear con datos válidos	Aparece en la lista	201 y el recurso creado
Crear con un Id o número ya existente	Mensaje de duplicado en el formulario	409
Crear con el nombre vacío o demasiado largo	Mensaje bajo el campo	400
Crear un teléfono con dueño inexistente	El desplegable solo ofrece personas existentes	400 con detalle del campo **`Duenio`**
Editar un registro	Los cambios se ven en la lista	204
`PUT` con clave distinta en la ruta y el cuerpo	–	400
`PUT` o **`GET`** sobre un registro inexistente	Página de no encontrado	404
Eliminar una profesión con estudios	Mensaje de bloqueo en la pantalla	409
Eliminar un teléfono	Desaparece de la lista	204

Eliminar una persona con teléfonos	Sus teléfonos desaparecen también	204

Para probar en Swagger: abrir **`/swagger`**, desplegar el endpoint, **Try it out**, escribir un cuerpo mínimo (sin campos de navegación) y **Execute**. Un cuerpo de ejemplo para Telefono:

```
{ "num": "3001234567", "oper": "Claro", "duenio": 1001 }
```

Despliegue local (resumen para el README)

1. Instalar SQL Server 2022 Express, SSMS y Visual Studio 2026 con .NET 10.
2. Ejecutar **`Scripts/01_ddl.sql`** en SSMS para crear **`persona_db`**.
3. Clonar el repositorio y abrir **`personapi-dotnet.sln`**.
4. Revisar que **`DefaultConnection`** en **`appsettings.json`** apunte a la instancia propia.
5. Compilar con **Ctrl + Shift + B** (debe decir 0 erróneo) y ejecutar con **F5**.
6. Abrir **`/Profesion`** y **`/swagger`** en el navegador.

Error al compilar con Ctrl + Shift + B: si Visual Studio muestra “Las herramientas de contenedor de Visual Studio requieren Docker Desktop”, es porque Ctrl + Shift + B compila toda la solución, incluido el proyecto **`docker-compose`**, que solo sirve para ejecutar con contenedores. Para la ejecución local basta compilar solo el proyecto web: clic derecho sobre **`personapi-dotnet`** en el Explorador de soluciones y **Compilar**; o bien, clic derecho sobre **`docker-compose`** y **Descargar proyecto**.

El repositorio también incluye archivos de Docker (el proyecto **`docker-compose`**); el procedimiento verificado en este laboratorio es la ejecución local descrita arriba, que no necesita Docker. Falta aún el script DML con datos de ejemplo y el tag de entrega.

Resumen de decisiones arquitectónicas

Decisión	Alternativa descartada	Razón
Monolito con MVC	Servicios separados	Cuatro tablas y tres personas: una sola aplicación es más simple de construir, probar y entregar.

DAO con interfaz y repositorio	Usar el **`DbContext`** directo en los controladores	Controladores desacoplados de la persistencia y fáciles de probar.
Database First	Code First	La base ya estaba definida; las clases coinciden con ella sin escribirlas a mano.
Dos controladores por entidad (MVC y API)	Un solo controlador para ambos	Cada uno tiene una salida distinta (vista o JSON) y comparten el repositorio.
ViewModels con validación	Exponer las entidades	Evita enviar campos de navegación y deja las reglas en un solo lugar.
`AddScoped` para repositorios y contexto	Singleton o instancias con **`new`**	Un contexto por petición, sin estado compartido entre usuarios.
Cadena de conexión en **`appsettings.json`**	Dejarla en **`OnConfiguring`**	Un solo lugar de configuración y sin datos de conexión dentro del código.
Borrado: cascada en Persona, bloqueo en Profesion	La misma regla para todas	La regla depende de quién depende de quién.
Ramas por entidad con integración en **`develop`**	Trabajar todos sobre **`main`**	Un cambio a medias no rompe la compilación de los demás.
.NET 10 y SQL Server 2022	.NET 6/7 y SQL S	

Conclusiones y lecciones

El proyecto permitió desarrollar una aplicación web monolítica para la gestión de personas y sus entidades relacionadas, integrando MVC, repositorios, SQL Server y Entity Framework Core. Se logró separar las responsabilidades de sus componentes, implementar operaciones CRUD y exponer servicios mediante una API REST documentada con Swagger. Además, el uso de Git y GitHub facilitó la organización y el trabajo colaborativo.

Durante el desarrollo se comprendió la importancia de separar responsabilidades mediante capas, utilizar interfaces y repositorios para organizar el acceso a los datos, y emplear ViewModels para manejar la información y sus validaciones. También se reforzó el manejo de Entity Framework Core, la integración con SQL Server y el uso de herramientas de control de versiones para coordinar el trabajo en equipo.

Referencias

[1] Microsoft, “Common web application architectures,” Microsoft Learn. [En línea]. Disponible en: https://learn.microsoft.com/en-us/dotnet/standard/modern-web-apps-azure-architecture/common-web-application-architectures. [Accedido: 3-nov-2026].

[2] S. Smith, “Información general de ASP.NET Core MVC,” Microsoft Learn. [En línea]. Disponible en: https://learn.microsoft.com/aspnet/core/mvc/overview/. [Accedido: 3-nov-2026].

[3] E. Hieatt y R. Mee, “Repository,” Patterns of Enterprise Application Architecture, Martin Fowler. [En línea]. Disponible en: https://martinfowler.com/eaaCatalog/repository.html. [Accedido: 3-nov-2026].

[4] Microsoft, “SQL Server technical documentation,” Microsoft Learn. [En línea]. Disponible en: https://learn.microsoft.com/en-us/sql/sql-server/. [Accedido: 3-nov-2026].

[5] Microsoft, “Reverse Engineering (Scaffolding),” Entity Framework Core Documentation. [En línea]. Disponible en: https://learn.microsoft.com/en-us/ef/core/managing-schemas/scaffolding/. [Accedido: 3-nov-2026].

[6] Microsoft, “Create web APIs with ASP.NET Core,” Microsoft Learn. [En línea]. Disponible en: https://learn.microsoft.com/en-us/aspnet/core/web-api. [Accedido: 3-nov-2026].

[7] OpenAPI Initiative, “OpenAPI Specification, Version 3.1.0,” 15-feb-2021. [En línea]. Disponible en: https://spec.openapis.org/oas/v3.1.0. [Accedido: 3-nov-2026].

[8] S. Chacon y B. Straub, Pro Git, 2.ª ed. Apress, 2014. [En línea]. Disponible en: https://git-scm.com/book/en/v2. [Accedido: 3-nov-2026].

[9] GitHub, “About repositories,” GitHub Docs. [En línea]. Disponible en: https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories. [Accedido: 3-nov-2026].